

■ PYTHON DICTIONARY METHODS — COLOURFUL HANDWRITTEN NOTES

Classroom + Real-Time IT / Cloud / Security Examples + Practice Labs

1. What is a Dictionary?

Definition

A **dictionary** stores data as **key : value** pairs. It uses `{ }`. Keys are unique, while values can repeat. Dictionaries are mutable, so data can be added, changed and removed.

```
server = {  
    "name": "web01",  
    "ip": "10.10.1.20",  
    "port": 443,  
    "environment": "production"  
}  
  
server_name = server["name"]  
print(server_name)
```

Real-time idea

Dictionaries are everywhere in automation: JSON/API responses, cloud resources, server configuration, IAM records, Kubernetes data and application settings.

2. Add & Update Dictionary Data

```
server = {"name": "web01", "port": 80}  
  
server["ip"] = "10.10.1.20" # add  
server["port"] = 443 # update  
  
print(server)
```

Remember

`dict[key] = value` can both ADD a new key and UPDATE an existing key.

3. Most Important Dictionary Methods ■

get()

Safely returns the value for a key. A default value can be supplied.

```
config = {"region": "ap-south-1", "env": "prod"}  
  
region = config.get("region")  
owner = config.get("owner", "Unknown")  
  
print(region)  
print(owner)
```

Real-time use

Safely read optional API/configuration fields.

setdefault()

Returns the value if the key exists. Otherwise it creates the key with the default value.

```
server = {"name": "web01"}

server.setdefault("status", "running")
print(server)
```

Real-time use

Create default configuration only when a setting is missing.

update()

Adds multiple key-value pairs or updates existing keys.

```
server = {"name": "web01", "port": 80}

server.update({"port": 443, "env": "prod"})
print(server)
```

Real-time use

Update application, server or cloud configuration.

keys()

Returns a view containing all keys.

```
server = {"name": "web01", "ip": "10.10.1.20"}

keys = server.keys()
print(keys)
```

Real-time use

Get all field names from an inventory or API response.

values()

Returns a view containing all values.

```
server = {"name": "web01", "port": 443}

values = server.values()
print(values)
```

Real-time use

Inspect configuration values.

items()

Returns key-value pairs.

```
server = {"name": "web01", "port": 443}
```

```
for key, value in server.items():  
    print(key, "=", value)
```

Real-time use

Very useful for looping through JSON/configuration data.

4. Remove Dictionary Data

pop()

Removes a specified key and returns its value.

```
server = {"name": "web01", "ip": "10.10.1.20", "port": 443}

old_ip = server.pop("ip")
print(old_ip)
print(server)
```

Real-time use

Remove a known configuration field and keep the old value.

popitem()

Removes and returns the last inserted key-value pair.

```
config = {"env": "prod", "region": "india", "tier": "web"}

last_item = config.popitem()
print(last_item)
print(config)
```

Real-time use

Process/remove the most recently inserted dictionary entry.

clear()

Removes all key-value pairs.

```
session = {"user": "vishwa", "token": "ABC123"}

session.clear()
print(session) # {}
```

Real-time use

Clear temporary data after processing.

copy()

Creates a shallow copy of the dictionary.

```
prod = {"region": "ap-south-1", "replicas": 3}

test = prod.copy()
test["replicas"] = 1

print(prod)
print(test)
```

Real-time use

Create a working copy without changing the original top-level dictionary.

5. fromkeys()

```
servers = ["web01", "web02", "db01"]

status = dict.fromkeys(servers, "unknown")
print(status)
```

Real-time use

Quickly create an inventory/status dictionary where many keys start with the same default value.

6. Dictionary Built-ins & Operators

Function / Operator	Purpose	Example
<code>len(d)</code>	Number of pairs	<code>len(server)</code>
<code>key in d</code>	Check key exists	<code>'ip' in server</code>
<code>key not in d</code>	Check key absent	<code>'password' not in config</code>
<code>del d[key]</code>	Delete known key	<code>del server['port']</code>
<code>d[key]=value</code>	Add/change value	<code>server['port']=443</code>

7. Real-Time JSON / API Example ■

```
api_response = {
    "status": "success",
    "server": {
        "name": "web01",
        "ip": "10.10.1.20"
    }
}

status = api_response.get("status")
server_name = api_response["server"]["name"]

print(status)
print(server_name)
```

Real-time

REST APIs commonly return JSON objects, which map naturally to Python dictionaries. You can read, update and build API payloads with dictionaries.

8. DevOps / Cloud / Security Examples

AWS Resource Tags

```
tags = {
    "Name": "web01",
    "Environment": "Production",
    "Owner": "Security"
}

environment = tags.get("Environment")
print(environment)
```

IAM User Record

```
user = {  
    "username": "vishwa",  
    "role": "admin",  
    "active": True  
}  
  
if user.get("active"):  
    print("User is active")
```

Kubernetes Configuration

```
deployment = {  
    "name": "payment-api",  
    "replicas": 3,  
    "namespace": "production"  
}  
  
deployment["replicas"] = 5  
print(deployment)
```

Security Finding

```
finding = {  
    "id": "CVE-2026-001",  
    "severity": "HIGH",  
    "status": "OPEN"  
}  
  
if finding.get("severity") == "HIGH":  
    print("Immediate action required")
```

9. Nested Dictionary

```
company = {
    "name": "VishwaTech-Labs",
    "team": {
        "security": 5,
        "devops": 8
    }
}

security_count = company["team"]["security"]
print(security_count)
```

Real-time

Nested dictionaries are common in JSON, REST API responses, cloud metadata and application configuration.

10. Dictionary Looping

```
servers = {
    "web01": "10.10.1.10",
    "web02": "10.10.1.11",
    "db01": "10.10.2.10"
}

for server, ip in servers.items():
    print(f"{server} -> {ip}")
```

Real-time

Use **items()** when you need both the key and value while looping through server inventories or API data.

11. Dictionary Comprehension

```
servers = ["web01", "web02", "web03"]

server_status = {
    server: "running"
    for server in servers
}

print(server_status)
```

Simple idea

Dictionary comprehension is a compact way to build a dictionary using an iterable and an expression.

12. Dictionary vs List vs Tuple vs Set

Feature	Dictionary	List	Tuple	Set
Stores	Key : Value	Values	Values	Unique values
Duplicates	Keys unique	Allowed	Allowed	Not allowed
Access	By key	By index	By index	Membership
Mutable	■	■	■	■

<i>Best use</i>	<i>Structured records</i>	<i>Changing sequence</i>	<i>Fixed sequence</i>	<i>Unique/comparison data</i>
-----------------	---------------------------	--------------------------	-----------------------	-------------------------------

13. Interview Questions ■

- What is a dictionary?
- What is a key-value pair?
- Can dictionary keys be duplicated? — No.
- Can dictionary values be duplicated? — Yes.
- Difference between `get()` and `dict[key]`?
- Difference between `pop()` and `popitem()`?
- What does `update()` do?
- What are `keys()`, `values()` and `items()` used for?
- Why use `copy()`?
- How do you check whether a key exists? — `key` in dictionary.
- How do you delete a key? — `pop()`, `del`, or `clear()`.
- Why are dictionaries heavily used with JSON and REST APIs? — JSON naturally represents key-value data.

14. Practice Labs ■

- Create a server dictionary with name, IP, port and environment.
- Use `get()` to safely read an optional key.
- Use `update()` to change a server port and add a new field.
- Use `pop()` and print the removed value.
- Use `keys()`, `values()` and `items()` in a loop.
- Create an AWS tag dictionary and print all tags.
- Create a nested Kubernetes deployment dictionary.
- Create an IAM user dictionary and check active status.
- Use `fromkeys()` to create a server-status dictionary.
- Build a JSON-like API response and extract nested values.

15. Quick Reference — Dictionary Methods

Method	Purpose
<code>get(key, default)</code>	Safely read a value
<code>setdefault(key, default)</code>	Get or create default
<code>update(other)</code>	Add/update pairs
<code>keys()</code>	Get keys
<code>values()</code>	Get values
<code>items()</code>	Get key-value pairs
<code>pop(key)</code>	Remove specified key
<code>popitem()</code>	Remove last inserted pair

<code>clear()</code>	<i>Remove all pairs</i>
<code>copy()</code>	<i>Shallow copy</i>
<code>fromkeys(keys, value)</code>	<i>Create dictionary from keys</i>

■ Final Rule

Dictionary = key : value.

Use dictionaries for structured records, JSON/API responses, cloud configuration, inventories and automation.

Core methods: `get()`, `setdefault()`, `update()`, `keys()`, `values()`, `items()`, `pop()`, `popitem()`, `clear()`, `copy()`, `fromkeys()`.